

Technical Report

Design, Implementation, and Verification of a Serial BCD Arithmetic Unit in Verilog

Reece M. Dobro

Abstract

A serial Binary Coded Decimal (BCD) arithmetic unit was designed, implemented, and verified using the Verilog hardware description language in AMD Vivado. The unit performs 4-digit BCD addition and subtraction using a serial 1-bit input stream and produces a 5-digit BCD result in serial form to accommodate potential carry-out. Functionality was validated through two simulation testbenches covering both typical operations and edge cases with analysis of the waveforms and the TCL console outputs that confirm that all results matched their expected values. In addition, RTL schematic analysis verified that the synthesized hardware aligned with the intended digital design. Overall, the simulation and RTL analysis results demonstrate that the serial BCD arithmetic unit operates correctly and satisfies all functional requirements specified by the project.

1 Introduction, Background, and Theory

The main objective of this project is to design, program, and test a serial BCD arithmetic unit. This unit is capable of performing 4-digit Binary Coded Decimal (BCD) addition and subtraction through a 1-bit serial bit-stream input, and outputs the computed result in 1-bit serial form. The computed result is 5-digits BCD in order to account for a carry-out value. Due to the complexity of the objective, the primary functions of the unit are separated into modules that are integrated into a top-level module, *Project3*. Due to the sequential nature of the unit, it utilizes an active-high reset signal.

Two simulations were created to evaluate and examine both of the modules, with a variation in inputs and delay between inputs to test the modules under various conditions. The results of the simulations were analyzed using waveform outputs and TCL console logs, which confirmed that the outputs of the module properly converted the inputs into their respective formats. In addition, the RTL Schematic from AMD Vivado was examined and compared to the digital design of the modules. Overall, the simulation results demonstrate that the design operates correctly and satisfies the functional requirements specified by the project specifications.

1.1 Binary Coded Decimal

A binary-coded decimal (BCD) value is a 4-bit binary number that can express values from 0 to 9. The value of a BCD digit is directly encoded in binary. For example, the number 7_{10} in BCD is written as 0111_2 . As the max value of base 10 decimal is 9, any value greater than 9 needs to be adjusted, or else it will be invalid (1010_2 to 1111_2). This correction occurs through the addition of 6_{10} and a carry-out value of 1. 6_{10} is added as there are 6 invalid values for BCD in a 4-bit binary number (10_{10} to 15_{10}). As each BCD digit is represented using 4 bits, an n -digit BCD value requires $4n$ bits in binary.

1.2 Inputs and Outputs

Signal	Direction	Size (bits)	Description
din	Input	1	Serial input for the unit
clock	Input	1	Free running clock governing timing
reset	Input	1	Active High Synchronous signal to bring unit to a known state
result	Output	1	Serial output for the unit

Table 1: Port Signal and Function Table for the Serial BCD Arithmetic Unit

The inputs, outputs, and expected behavior were provided by the project specifications for this assignment. The inputs and outputs for the top level module for the project are shown in Table 1. The serial input, *din*, and serial output, *result*, are covered in depth in the following sections.

1.3 Serial Input Stream for *din*

The serial input stream for the unit is fully defined by the project specifications. It starts with the control signal, 8'h67 (01100111 in binary), where the MSB (most significant bit) is given first. Following the control signal is the operand bit, which controls whether the arithmetic unit adds or subtracts the two 4-digit BCD values. If the value of the operand bit is 1'b1, then subtraction occurs; if the value is 1'b0, then addition occurs. Next, the first 4-digit BCD value is given, which, like the control signal, is given MSB first. The first 4-digit BCD value is also referred to as *A* in this paper. Following the first 4-digit BCD value is the second BCD value, which is also given MSB first. The second 4-digit BCD value is also referred to as *B* in this paper. In total, the input stream is 41 bits in length as there are 8-bits for the controls signal, 16 bit for each 4-digit BCD value, and 1 bit for the operand.

1.4 Serial Output Stream for *result*

The serial output stream for the unit is fully defined by the project specifications. It starts with the control signal, 8'hA5 (10100101 in binary), where the MSB (most significant bit) is given first. Following the control signal is the result of the computation of the 4-digit BCD value, *A*, and the 4-digit BCD value, *B*. The result is 20 bits in length due to being a 5-digit BCD value, which is sized to account for an additional BCD digit for the carry out. In total, the serial output stream is 28 bits in length as there are 8-bits for the controls signal and 20 bits for the 5-digit BCD result. As the serial input stream is larger in size then the serial output stream, inputs can continuously be fed into the output without any errors.

1.5 Supporting Modules

Module Name	Function
SIPO_41	Converts 1-bit serial input into 41-bit registered output
PISO_28	Loads 28-bit parallel input into a register and converts to 1-bit serial
BCD_1D	Performs 1-digit BCD addition with 1-bit carry-in and carry-out
BCDadd_4d	Performs 4-digit BCD addition with 1-bit carry-in and 1-digit BCD carry-out
BCD_4D_C	Performs 4-digit BCD subtraction
BCD_ALU	Computation logic for adding and subtracting 2 4-digit BCD numbers with a operand input
counter_41	Counter that increments on each clock cycle up to 41 cycles
dff	D Flip-Flop used as a 1-bit register
mirror_8bit	Reverses bit order of 8-bit binary input, mapping LSB to MSB
mirror_16bit	Reverses bit order of 16-bit binary input, mapping LSB to MSB
reg8	8-bit register
reg28	28-bit register
reg41	41-bit register

Table 2: Other modules used in the design

The top-level module, *Project3*, utilizes instances of several modules that help to manipulate, store, and select data. As mentioned previously, this was done due to the complexity of the design and provides several advantages. Specifically, this modular approach improves portability, enhances readability, and simplifies debugging. In addition, it allows for previously created modules to be easily integrated into this project, which simplifies the Verilog implementation process for the unit. Table 2 contains a list of modules used in this design, aside from the top-level module, *Project3*.

Figure 1 shows the architectural diagram of the 1-digit BCD adder. As shown in Figure 1, the design consists of two binary adders, two comparators, and a 2-to-1 multiplexer. The process of the design consists of first adding the two operands and carry-in value through binary, checking to see if the sum of that addition is an overflowed value, and adjusting accordingly if so. As mentioned previously, if a BCD digit overflows, it will need to have 6 added to it and have a carry-out value of 1. Additionally, if the intermediate result exceeds 9 (1001_2), even without producing a carry, 6 must still be added to obtain a valid BCD digit.

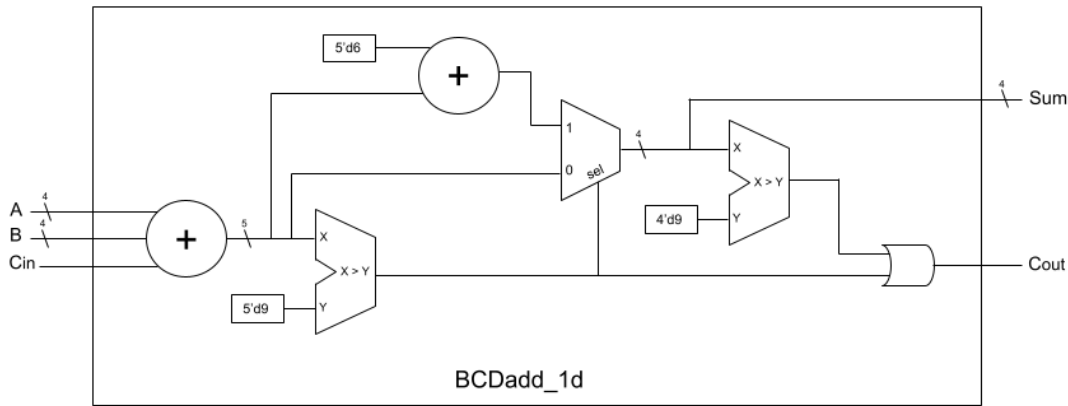


Figure 1: Architectural Diagram of the 1-digit BCD adder

Figure 2 shows the architectural diagram of the 4-digit BCD adder. As shown in Figure 2, the design consists of 4 instances of the 1-digit BCD adder, *BCDadd_1d*. The carries of the 4 instances are ripple carried, with the initial carry being provided by the 1-bit input, *carry_in*. The design of this module is similar to a ripple carry adder (RCA), with full adder instances being used, and 1-digit BCD adders are. In addition, each instance of the 1-digit BCD adders inputs and outputs four bits, unlike the full adders for an RCA design that have one bit.

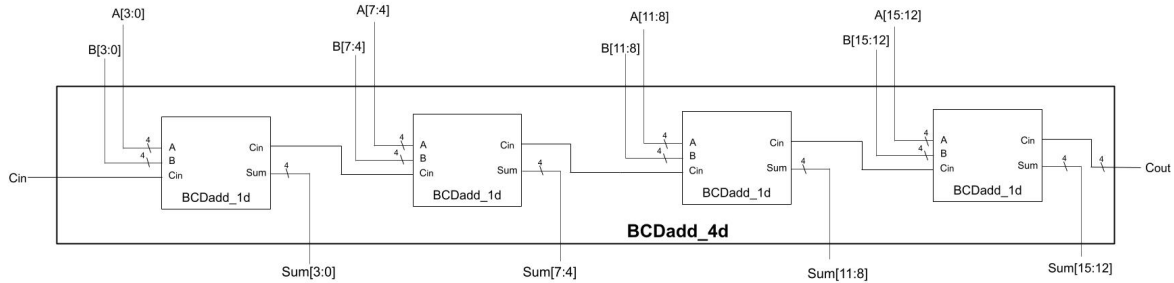


Figure 2: Architectural Diagram of the 4-digit BCD adder

As shown by Table 2, *BCD_ALU*, either adds or subtracts two 4-digit BCD values, with the operation depending on a 1-bit input, *operand*; therefore, either $(A+B)$ or $(A-B)$ will be calculated. The design of the BCD ALU and the instances used by it consist of only combinational logic. Figure 3 shows the architectural diagram for the module.

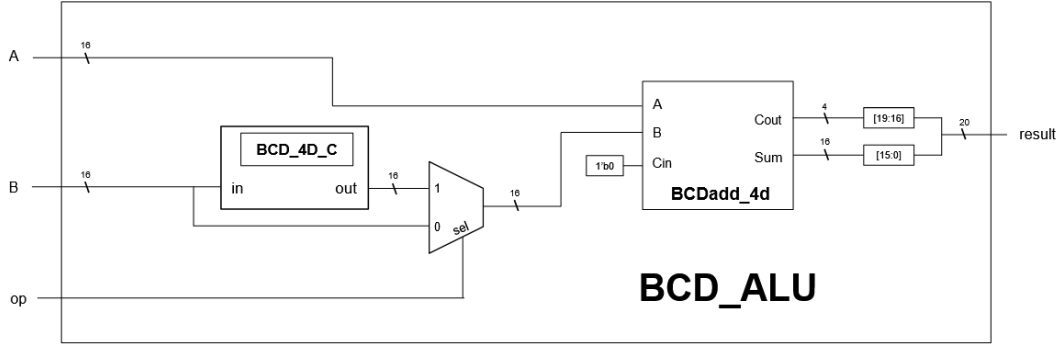


Figure 3: Architectural Diagram of the Module, *BCD_ALU*

As shown in the figure, *op* controls what the “B” input for the 4-digit BCD adder receives through the use of a 2-to-1 multiplexer. When the operand bit is low, the adder will receive the value “B” given to the input port *B* of the ALU. When the operand bit is high, the adder will receive the complement value of “B”, which can be referred to as -B. The complement given through an instance of the *BCD_4D_C* module. The design of the *BCD_4D_C* module is shown in Figure 4. As shown in the figure, the 4-digit BCD input is separated, where each BCD digit is subtracted by 4’d9. As a BCD digit represents a decimal value from 0 to 9, 9 is used because it is the maximum digit in base-10, allowing the 9’s complement to be found. In order to complete the negation, 1’d1 must be added to the 9’s complement. The *BCDadd_4d* instance does this, adding the 9’s complement version of the 4-digit BCD number with 1 to produce an output that is the full negated 4-digit BCD version of the input. Not shown in the architectural diagram is that if the operation is subtraction (*op* is 1'b1), the BCD carry value will be assigned to a BCD value of 0, as in binary subtraction, the final carry bit in BCD subtraction is discarded.

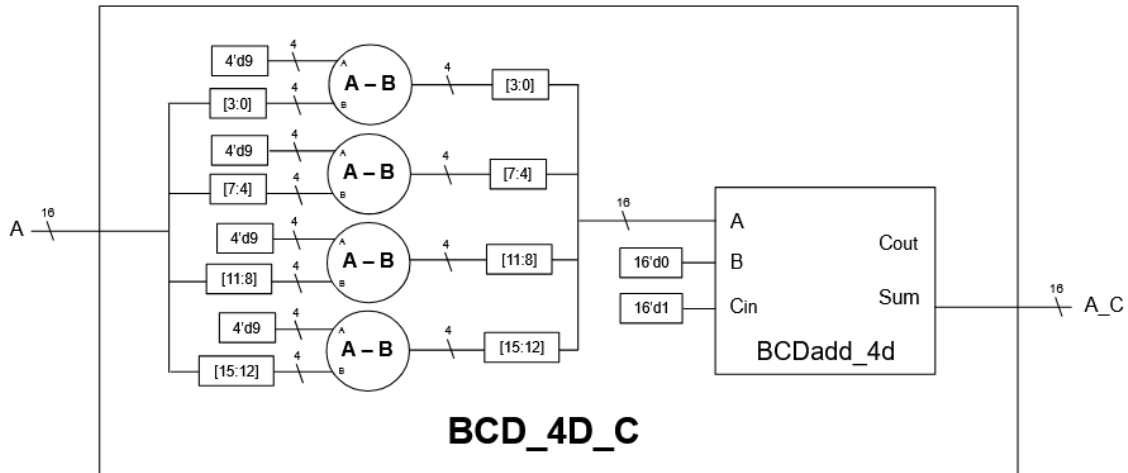


Figure 4: Architectural Diagram of the Module, *BCD_4D_C*

The module, *BCDadd_4d*, is a ripple carry adder made up of four instances of *BCDadd_1d*. The 1-digit BCD adder, *BCDadd_1d*, accepts two BCD operands and a 1-bit carry-in value. This is performed by adding the operands and carry-in value in binary, then converting the sum to a valid BCD sum with a carry-out value. For this single-digit BCD adder, it converts numbers when the initial binary addition produces a carry-out or if the intermediate 4-bit sum falls within the invalid range. The 4-digit BCD adder has four instances of the 1-digit BCD adders that are ripple-carry to create a 4-digit BCD

adder. This is done by connecting the carry out of each individual stage to the carry-in of the next BCD adder stage. The operand inputs for each stage are mapped to corresponding 4-bit segments of the 16-bit input vectors.

The module, *counter_41*, is a 41-loop counter. When the 1-bit input, *start*, is high (on a clock posedge), the counter begins. The 1-bit output, *active*, is high only while the counter is incrementing. After 41 cycles, *active* becomes low and the counter is reset to 0. While the counter is incrementing, *start* being triggered has no impact on the output or the state of the module. Figure 5 shows the design of the module, which utilizes both a *dff* instance and *reg8* instance. *reg8* contains the current value of the counter, which, along with the *dff* instance, is reset to 0 when either the external reset is driven high or the value of the counter reaches 8'd40 internally. It is not 8'd41 as there is a 1 clock cycle delay between a change in *start* and a change in incrementation. *reg8* has been 6 bits in length as the maximum value the register would need to hold is 40, and a 6-bit register can represent $2^6 = 64$ distinct values. *reg8*, however, was used as it was a preexisting module. The *dff* instance is used to continue the increment of the counter by 1 even when the 1-bit input, *start*, is 1'b0.

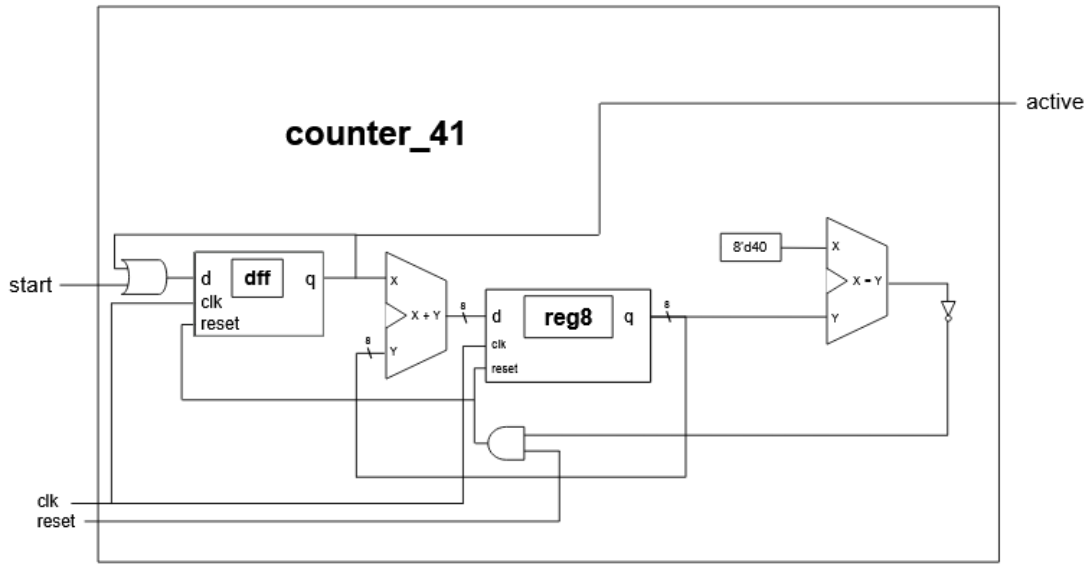


Figure 5: Architectural Diagram of the Module, *counter_41*

The module, *SIPO_41*, is a Serial Input Parallel Output unit that is designed to take in the 1-bit serial input and assign it into a 41 bit register. 41 bits is the length of the register instance, as that is the length of the serial input stream for the project. The main input of the module, *serial_in*, is assigned to the MSB of the 41-bit register, and the LSB contains the bit input 41 clock cycles previously. Due to the sequential nature of the unit and the use of a register, a clock and reset signal are given as inputs into the module.

The module, *PISO_28*, is a Parallel Input Serial Output unit that is designed to take in a 28-bit parallel input, *parallel_in*, and output it as a 1-bit serial signal. The size of the input is 28-bits as that is the size of the serial output stream, discussed in section 1.5. The module's 1-bit serial output is the MSB bit of the 28-bit register output. The 28-bit parallel input is loaded into a 28-bit register input when the 1-bit input, *load*, is high. When the 1-bit input, *load*, is low, the module doesn't assign the 28-bit parallel input, *parallel_in*, to the 28-bit register input. Instead, it assigns the input of the register with a 1-bit left-shifted value from the register's own output, with 1'b0 assigned to the LSB. This design allows the 28-bit parallel input to be output in MSB-first serial form after 28 clock cycles, while also storing the value. Due to the 1'b0 assigned to the LSB, after the module completely outputs the 28-bit input, it will begin to output 1'b0 until a new value is loaded in. Due to the sequential nature of the unit and the use of a register, a clock and reset signal are given as inputs into the module.

1.6 Top Level Module Design

Figure 6 shows the architectural diagram of the top-level module of the project, *Project3*. The module uses instances of the supporting modules to execute the behavior outlined in the project specification. Every cycle, the module will add the 1-bit serial input to a 41-bit parallel output register. During that cycle, it will also mirror bits [7:0] and compare with 8'h67; if it is a match, then the 41 cycle counter will start and its one bit output, *active*, will be high on the next cycle. On the current cycle, before *active* is high, the “A” and “B” will be given as inputs to the BCD ALU by taking them from the parallel register of the SIPO and mirroring the bits, with the MSB becoming the LSB and vice versa. The operand bit will also be taken from the register and input into the BCD ALU to dictate whether addition or subtraction will occur. As there are no proportional delays and the BCD ALU is made up of entirely combinational logic, the correct result for the inputs will be given during the same clock cycle that the inputs are given. This result is concentrated with the output control signal, 8'hA5, and input into the input for the PISO module. As the input control signal comparison output (delayed by a cycle due to the dff module) and the active counter bit are high, the PISO module will load the 28-bit combined control signal and result value into its register. After 28 cycles, the final bit of the result, the LSB, will be output. After 14 more cycles, the counter will be internally reset, and the PISO module will be able to be loaded with a new result.

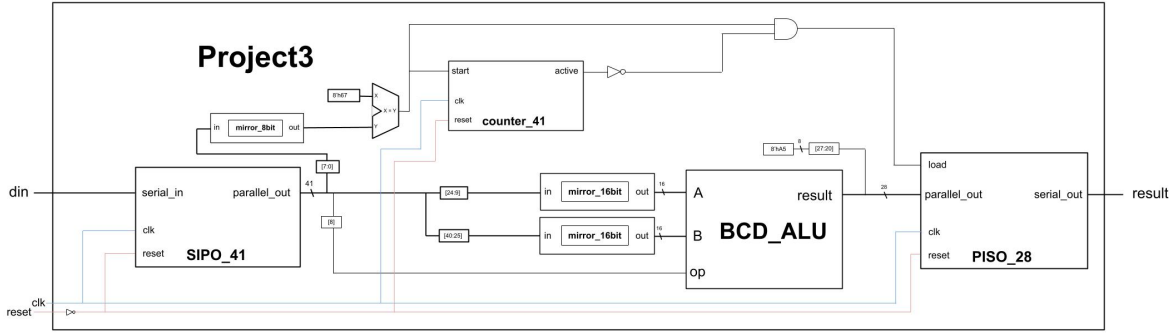


Figure 6: Architectural Diagram of the Top Level Module, *Project3*

As shown from the Figure 6, not all of the supporting modules are used in the top level module, *Project3*. Instead, several of the modules are reused within higher-level hierarchical designs. For example, *BCD_1D* is instantiated within *BCD_add4* as the unit utilizes a ripple-carry design. Similarly, *BCD_add4* and *BCD_4D_C* are contained in the BCD arithmetic unit, *BCD_ALU*. A critical error that had to be accounted for was the control signal being falsely activated during the serial input stream due to either “A” or “B” containing the value 8'h67. This has been prevented by having result value only loaded into the PISO register when *counter_41* isn’t active and the SIPO output contains 8'h67. Also, the not gate used on the reset red line, shown in Figure 6, is present due to an active high reset being required by project specifications. As all supporting modules were designed for an active low reset signal, this had to be accounted for.

1.7 Verilog Implementation

The Verilog implementation of the top level module of the project, *Project3*, is shown in Figure 7. The source code of the module is based on the architectural diagram from Figure 6. Due to the use of instances in the Verilog implementation, several wires are utilized. The naming of the wires reflects their function. For example, the wire *flip_A* serves as the net between the output of the mirrored value of the BCD value, A, and the BCD ALU unit. While certain nets could have been simplified, such as *report*, they are used to make the code more readable and easier to debug.

```

module project3(
    input din,
    input reset,
    input clock,
    output result,
    output load,
    output active
);
    wire [40:0] p_out;
    wire [7:0] flip_in;
    wire [15:0] flip_a;
    wire [15:0] flip_b;
    wire start;
    wire [19:0] result2;

    SIPO_4l u1(din, clock, !reset, p_out);
    mirror_8bit u2(p_out[7:0], flip_in);
    assign start = (flip_in==8'h67);
    counter_4l u3(start, !reset, clock, active);
    mirror_16bit u4(p_out[24:9], flip_a);
    mirror_16bit u5(p_out[40:25], flip_b);
    BCD_ALU u6(flip_a, flip_b, p_out[8], result2);
    assign load = start & !active;
    PISO_28 u8({8'hA5, result2}, load, clock, !reset, result);
endmodule

```

Figure 7: Source Code of Module *Project3*

The modules *dff*, *reg8*, *reg28*, and *reg41* were written at the behavior level. All other modules were written at the dataflow level.

2 Simulation, Results and Analysis

2.1 Simulation Details

To verify the accuracy of the serial BCD arithmetic unit, two simulations through AMD Vivado will be executed. Each simulation will consist of two 4-digit BCD values being entered into the serial input stream, followed by a second set of two 4-digit BCD values. The outputs of the top level module and its behavior will be analyzed to determine if it is successful. To ensure the simulation accurately reflects the intended operation of the unit, all tests are conducted under the following conditions.

- The time unit of each simulation will be 1 ns.
- The period of the 1-bit input *clock* will be 10 ns.
- Before the start of the first test case, *reset* will be set high for 6 ns, then remain low for the rest of that test case's simulation.
- All input packets begin with the required control header 8'b01100111
- Simulation will stop after the second test vector's final serial bit output is given on the positive clock edge
- Inputs for each test case are strictly limited to the defined test case vectors given in Table 3 and Table 4.

2.2 Test Cases

Table 3 shows the test values used for the first simulation, and Table 4 shows the test values used for the second simulation. The operands and expected results are given as hexadecimal values; however, for the simulation, they will be inputted into the top level module using the serial input stream format and output as the serial output format. Hexadecimal formatting for the respective values was given for readability. The first set of test vectors listed in Table 3 is a maximum value subtraction test.

Test Case	Op Bit	Operands [15:0]		Expected Result (hex)
		A (hex)	B (hex)	
1	1'b1 (-)	16'h9999	16'h0000	20'h09999
2	1'b0 (+)	16'h1234	16'h8765	20'h09999

Table 3: Simulation 1: Serial BCD ALU Test Cases

The largest 4-digit BCD input (16'h9999) is given for operand A, and the smallest 4-digit BCD input (16'h0000) is given for operand B in this test. This test is used to verify the correct operation of the subtraction logic under maximum input conditions. The second set of test vectors listed in Table 3 is used to verify the correct BCD addition under full-digit carry propagation. It ensures that the BCD ALU module properly handles carries across all four BCD digits and produces the correct 5-digit result. It also checks that the final carry-out bit (bit 17) is correctly generated. In addition, it confirms that no incorrect carries are produced when they should not occur. For the first simulation, there will be a 40ns delay between the first test case and the second test case serial input streams. During the delay, a constant stream of 1'b0 will be given.

Test Case	Op Bit	Operands [15:0]		Expected Result (hex)
		A (hex)	B (hex)	
1	1'b1 (-)	16'h6767	16'h6767	20'h00000
2	1'b0 (+)	16'h6767	16'h6767	20'h13534

Table 4: Simulation 2: Serial BCD ALU Test Cases

The second simulation uses identical 4-digit BCD operands (16'h6767) for both operand A and operand B in both addition and subtraction test cases. By having (16'h6767) be the operand, it tests the module's ability to differentiate between an operand for the BCD adder and the control signal. By having the control signal (8'h67) appear numerous times throughout the simulation, it ensures that the counter is properly incrementing. This test is used to verify the correct operation of the BCD ALU under symmetric input conditions. For subtraction, it validates that the module correctly produces a zero result when both operands are equal and also ensures that the correct final carry-out is produced (4'h0). For the second test case of simulation 2, it confirms correct BCD addition and verifies that carries are handled correctly when intermediate digit sums exceed 9. In addition, it ensures that the BCD adder is able to convert invalid BCD numbers into their correct counterparts with a carry. For the second simulation, there will be no delay between the first test case and the second test case serial input streams. This is done to ensure the system can handle a constant serial input stream and that non-overlapping sequences are detected.

To execute the test cases, two testbenches were created in Verilog. The simulation will output its critical information through both the TCL Console and waveforms. The waveforms output ensures that ports are mapped according to design specifications and can help to debug any issues discovered during testing. The TCL Console displays outputs in a text-based format that allows for simpler analysis of the ports.

2.3 Simulation Results

Table 5 contains the results of the first simulation compared to the expected results. The TCL Console output from the first simulation was compiled into Table 5 to show the actual results and allow for direct comparison to the expected results. Hexadecimal formatting for the respective values was given for readability. As mentioned before, the first bit of the output stream is the MSB of the control bits, 8'hA5. The full waveforms simulation is shown in Figure 8. Due to the length of the simulation, Figure 9 and Figure 10 contain enhanced waveforms of the outputs for each test case. As shown by Table 5, the outputs of the serial BCD arithmetic unit exactly match the expected behavior. This verifies that the functions tested by the test cases are working correctly and in accordance with project specifications.

Test Case	Op Bit	Operands [15:0]		Expected Result (hex)	Actual Result (hex)
		A (hex)	B (hex)		
1	1'b1 (-)	16'h9999	16'h0000	20'h09999	20'h09999
2	1'b0 (+)	16'h1234	16'h8765	20'h09999	20'h09999

Table 5: Simulation 1 Results Comparison

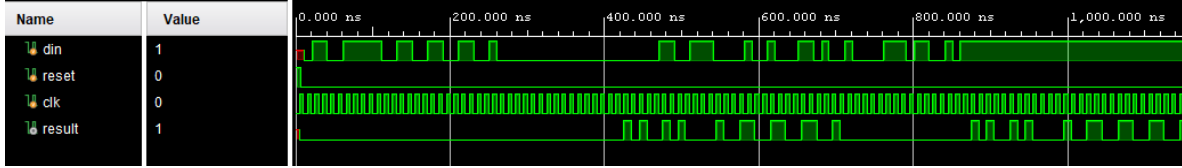


Figure 8: Full Waveforms Output of First Simulation

The total duration of the 1st simulation is 1,155 ns, as shown in Figure 8. This includes the inputs of both test cases and the full output of both test cases. As both outputs are correct, it can be concluded that the serial BCD arithmetic unit can handle a delay in the input streams for 2 separate operations. Also, a reset signal wasn't needed for the module to process in the second test input stream, which indicates that all storage elements were properly cleared.

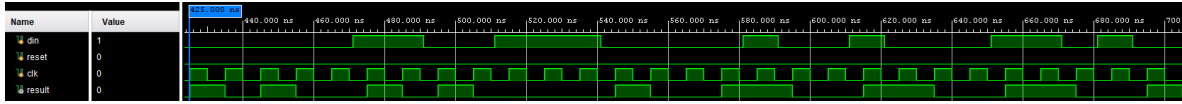


Figure 9: Waveforms Output of First Test Case of the First Simulation

The output of the first test case of the first simulation, as shown in Figure 9, occurred from 425 ns to 705 ns. This is determined as the first bit of the control signal for the first test case output occurs 425 ns from the start of the simulation. As the output bit updates on every positive edge of the clock, the clock's period is 10ns, and the length of the 1-bit serial output stream is 28 bits, the output of the first test case ends at 705 ns. As shown in Figure 9, the most significant BCD digit of the output is 0. This is the correct result as the sum of the most significant BCD values of the two operands is less than 10. This determines that the ALU carry logic for addition is functioning correctly. However, the discard logic outputs a value of 0 due to the operation bit being 1, indicating subtraction. The output of the simulation after the output control signal is given is 20'h09999, as shown by the figure. This demonstrates that the serial BCD arithmetic unit is able to do correct BCD addition for a typical operation.

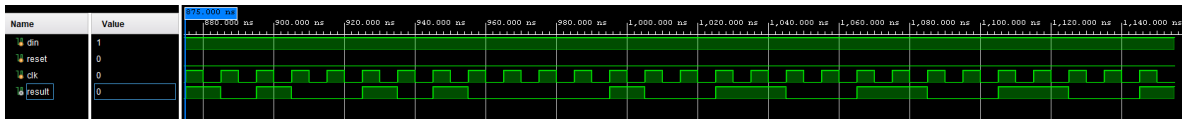


Figure 10: Waveforms Output of Second Test Case of the First Simulation

The output of the second test case of the first simulation, as shown in Figure 10, occurred from 875 ns to 1,155 ns. This is determined as the first bit of the control signal for the second test case output occurs 875 ns from the start of the simulation. As the output bit updates on every positive edge of the clock, the clock's period is 10ns, and the length of the 1-bit serial output stream is 28 bits, the output of the second test case ends at 1,155 ns. As shown in Figure 10, the most significant BCD digit of the output is 0. This is the correct result as the sum of the most significant BCD values of the two operands is less than 10. This determines that the ALU carry logic for addition is functioning correctly. The output of the simulation after the output control signal is given is 20'h09999, as shown by the

figure. This demonstrates that the serial BCD arithmetic unit is able to do correct BCD addition for a typical operation.

Table 6 contains the results of the second simulation compared to the expected results. Hexadecimal formatting for the respective values was given for readability. The TCL Console output from the second simulation was compiled into Table 6 to show the actual results and allow for direct comparison to the expected results. The full simulation is shown in Figure 11. Due to the length of the simulation, Figure 12 and Figure 13 contain enhanced waveforms of the outputs for each test case. As the actual result of the simulation matches the expected behavior, it can be concluded that the functions tested by the two test cases, such as maximum value subtraction for the BCD ALU, are working correctly in accordance with project specifications.

Test Case	Op Bit	Operands [15:0]		Expected Result (hex)	Actual Result (hex)
		A (hex)	B (hex)		
1	1'b1 (-)	16'h6767	16'h6767	20'h00000	20'h00000
2	1'b0 (+)	16'h6767	16'h6767	20'h13534	20'h13534

Table 6: Simulation 2: Serial BCD ALU Test Cases

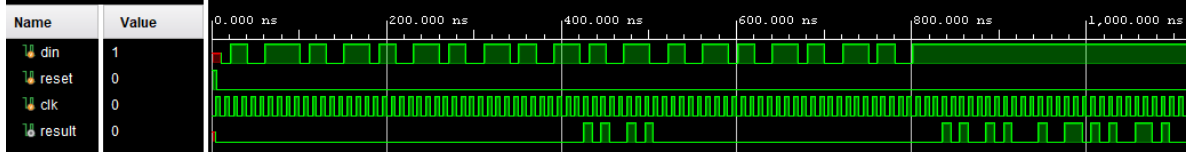


Figure 11: Full Waveforms Output of Second Simulation

The total duration of the 2nd simulation is 1,115 ns, as shown in Figure 11. This includes the inputs of both test cases and the full output of both test cases. As both outputs are correct and didn't overlap, it can be concluded that the serial BCD arithmetic unit can handle a continuous input stream. Also, a reset signal wasn't needed for the module to process in the second test input stream, which indicates that all storage elements were properly cleared. Furthermore, the input stream of the simulation contained the control signal, 8h'67, 8 times, and still had the correct output. This demonstrates that 41-bit counter logic is executing appropriately and that the module is able to differentiate between a control signal and an operand.

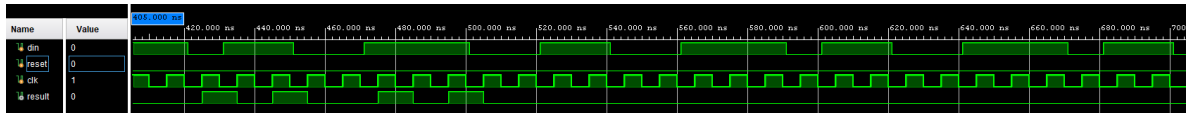


Figure 12: Waveforms Output of First Test Case of the Second Simulation

The output of the first test case of the second simulation, as shown in Figure 12, occurred from 425 ns to 705 ns. This is determined as the first bit of the control signal for the first test case output occurs 425 ns from the start of the simulation. As the output bit updates on every positive edge of the clock, the clock's period is 10ns, and the length of the 1-bit serial output stream is 28 bits, the output of the first test case ends at 705 ns. As shown in Figure 12, the most significant BCD digit of the output is 0. This determines that the overflow value logic for subtraction is occurring correctly. As the input operands are being subtracted and are the same value, a carry value of 1 is propagated through the ALU; however, the discard logic outputs a value of 0 due to the operation bit being 1, indicating subtraction. The output of the simulation after the output control signal is given is 20'h00000, as shown by the figure. This demonstrates that the serial BCD arithmetic unit is able to do correct BCD subtraction at a critical edge case.

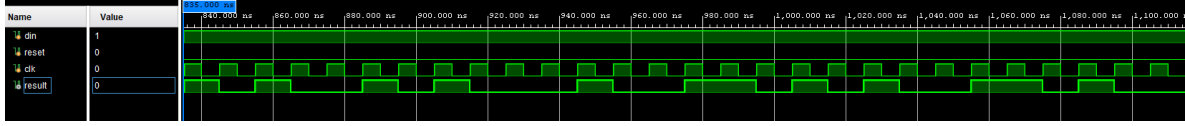


Figure 13: Waveforms Output of Second Test Case of the Second Simulation

The output of the second test case of the second simulation, as shown in Figure 13, occurred from 835 ns to 1,115 ns. This is determined as the first bit of the control signal for the second test case output occurs 835 ns from the start of the simulation. As the output bit updates on every positive edge of the clock, the clock's period is 10ns, and the length of the 1-bit serial output stream is 28 bits, the output of the second test case ends at 1,115 ns. As shown in Figure 13, the most significant BCD digit of the output is 1. As the sum of the operands is greater than 10,000 and the operand bit for test case 2 is 1'b0, which indicates addition, it can be concluded that 1'b1 is the correct value for the most significant BCD digit of the output. As a result, it can be determined that the overflow value logic for addition is working correctly. The output of the simulation after the output control signal is given is 20'h13534, as shown by the figure. This demonstrates that the serial BCD arithmetic unit is able to do BCD addition with carry propagation and generation.

Figure 14 provides an expanded view of the output waveform of the second simulation between 0 ns and 40 ns. This outcome is the same for the first simulation as both simulations begin with a 6 ns reset, have the same clock cycle, and have the same control signal input. The figure shows the value of the 1-bit serial input, *din*, being 1'bX. This is also shown on Figures 8 and 11, though not as clearly. As shown in Figure 14, the 1-bit serial input, *din*, is registered from a value of X to a known value at 11 ns. This occurs because from 0 ns to 11 ns, *clock* and *reset* are the only inputs being driven by the testbench. It is not until 11 ns that *din* is first defined in the testbench. This is done as a reset needs to occur before the design can take in the first value for the other inputs.

Figure 14 also shows the value of output *result*, being 1'bX at the start of the simulation. The output remains X until 5 ns, where it reflects its expected value. This occurs due to the design of the system. The output *result* is driven by the LSB of the 28-bit register instantiated by the PISO module. When a register is first initialized in the simulation, it has a value of X, which is why the outputs begin the simulation with a value of X. The registers for the design use D Flip-Flops to assign new values, and D Flip-Flops are updated on the positive edge of the clock signal. As the clock signal's positive edge occurs at 5 ns into the simulation, from 0 ns to 5 ns, the value of the registers that drive the outputs is unknown. In order to have the outputs be defined for the entirety of the simulation, the clock signal would need to begin high, not low. This initial delay is an inherent characteristic of D Flip-Flop logic rather than a functional error. From this analysis, it can be concluded that the initial unknown state of the input, *din*, and the output, *result*, is expected behavior rather than a result of a functional error in the design.

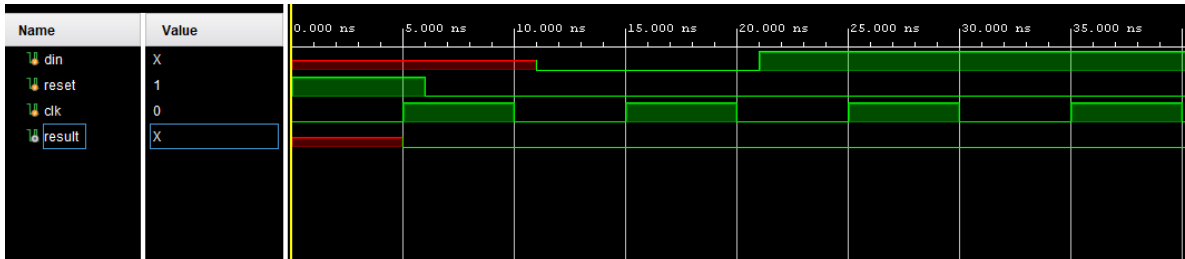


Figure 14: Enhanced Waveform of the Second Simulation from 0 ns to 40 ns

As the results of the simulations match the expected outputs and the waveforms of the simulations show the correct behavior of the serial BCD arithmetic unit, the Verilog implementation and the architectural design are deemed successful.

A potential limitation in the verification process is that only a limited number of test cases were utilized. For a 4-digit BCD value, there are $10^4 = 10,000$ possible input combinations. With two 4-digit BCD values and an operand bit, that comes to a total of $10^4 \times 10^4 = 100,000,000$ values. Since the value of the first operand must be equal to or greater than the second operand due to the

project specification, there are a total of $2 \times \frac{N(N+1)}{2} = 10000 \times 10001 = 100,010,000$ possible valid input combinations. While the simulation scenarios cover the primary operations and edge cases, they represent only a fraction of the total input space. However, it is impractical to simulate every possible valid value due to the time required for manual verification and documentation. This is why representative test cases, specifically boundary cases and typical operations, were selected to confirm the design's success under a variety of conditions.

2.4 RTL Schematic Results and Comparison

Figure 15 shows the schematic of the RTL Analysis from AMD Vivado for the top level module for the project. These figures provide a view of the synthesized design as interpreted by AMD Vivado. The Verilog program written in AMD Vivado is translated into basic hardware components such as multiplexers and logic gates. Due to design complexity, Vivado collapses hierarchical modules in the RTL schematic, limiting visibility of internal submodules in the report view.

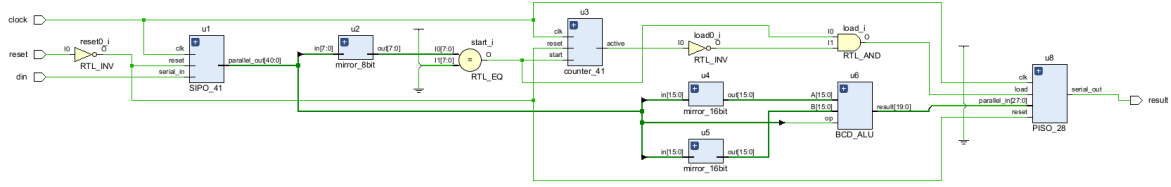


Figure 15: RTL Schematic result for the top-level module, *Project3*

The schematic of the serial BCD arithmetic unit, Figure 15, matches closely with the design from the architectural diagram created for the top-level module, Figure 6. Both figures include the NOT gate to turn the 1-bit active high *reset* into active low. They also both feature the AND gate and NOT gate that drive the *load* input for the *PISO_28* module. All module instances shown in the architectural design, such as *PISO_28*, are shown as blue boxes in Figure 15. Due to the close similarity between these designs, it can be concluded that the Verilog implementation of the architectural diagram was successful.

3 Conclusions

In this lab, the correct design, implementation, and verification of a serial BCD arithmetic unit was achieved. The simulation results confirmed that the unit functions appropriately and was able to correctly extract BCD digits from a serial input stream, perform either addition or subtraction, and then output the result as a 5-digit BCD sum with a control signal. Simulation results also confirmed that the module can handle BCD arithmetic under both typical operation and edge cases. The implementation was shown to be successful, with the RTL schematic matching the digital design. Future work could include resource analysis to identify potential hardware modifications that would improve overall efficiency. Overall, this lab demonstrates how Verilog can be used to implement and verify arithmetic modules digitally, producing modular designs without requiring physical components or additional hardware.

This report was compiled using L^AT_EX .